30.04.2026

OPENCAMPUS.sh

Applied Machine Learning

DATA MANAGEMENT AND NEURAL NETWORK COMPONENTS

https://de.pinterest.com/pin/302937512409503821/

EVENT NOTE: NIGHT OF THE TALENTS

- **<u>When:</u>** May 4th, 2026 @ 17:00 (next Monday)
- **<u>Where:</u>** InnoPier, Martensdamm 6, 24103 Kiel

EVENT NOTE: NIGHT OF THE TALENTS

- **<u>When:</u>** May 4th, 2026 @ 17:00 (next Monday)
- **<u>Where:</u>** InnoPier, Martensdamm 6, 24103 Kiel

https://ahoi.kiel-sailing-city.de/de/kiel-sailing-city/streaming/detail/POI/p_100244627/innopier

- Quiz
- Assignment Presentation & Recap
- Projects
- Optimizers & Gradient Descent
- Summary
- Outlook: Next Week

QUIZ

https://forms.office.com/r/j7rCvjvDvX

ASSIGNMENT PRESENTATION

ASSIGNMENT RECAP

ASSIGNMENT RECAP

- Introduction to `Dataset()` and `DataLoader()`
- Splitting of Data into Training, Validation and Test Subsets
  - Distribution Shift
  - A good Split preserves the Full Distribution
- Batching of Data
- Convolutional Neural Networks (Convolutional Blocks + MLP)
  - **Feature Extraction** and **Architectural/Automated Feature Engineering** via Conv. Blocks
- Logits: Raw Outputs for Classification Tasks
  - For Regression Tasks, we usually do not use the term "Logit"
- Modular Model Design
- Filters (Kernels) vs. Feature Maps
- Bonus: Models have no understanding of the Real World

PROJECTS

PROJECT MILESTONES

23.04.	Present your Ideas
30.04.	Form Groups & Put your Name in the Excel
07.05.	Literature Review
14.05.	Dataset Characteristics and Individual Feedback Sessions (**PUBLIC HOLIDAY**)

Deadline for completing the repo sections: 17.05.

21.05.	Definition of Model Evaluation
28.05.	Project Work
04.06.	Baseline Model Estimation

Deadline for completing the repo sections: 07.06.

11.06.	Project Work / Optional Feedback Sessions (**NO REGULAR CLASS**)
25.06.	Project Presentations, Part I
02.07.	Project Presentations, Part II

Submission deadline for the documented repo: 01.08.2026

PROJECTS: LITERATURE REVIEW

- Use the GitHub template to set up your project
- Review the relevant literature for your project
  - Use pages and tools like arxiv.org, ResearchGate, Google Scholar or SciSpace
- Select three sources and summarize them in your GitHub repository

OPTIMIZERS & GRADIENT DESCENT

OPTIMIZERS: STOCHASTIC GRADIENT DESCENT

- Optimizers define the **update rule** for the **parameters** θ of the model and adjust them **based on gradients of loss function**
- `optim.SGD` uses the **update rule** $\theta_{i,\text{new}} = \theta_{i,\text{old}} - \gamma \frac{\partial}{\partial \theta_i} J(\theta)$ with the **learning rate** γ and the **cost function** $J(\theta)$

OPTIMIZERS: STOCHASTIC GRADIENT DESCENT

- Optimizers define the **update rule** for the **parameters** θ of the model and adjust them **based on gradients of loss function**
- `optim.SGD` uses the **update rule** $\theta_{i,\text{new}} = \theta_{i,\text{old}} - \gamma \frac{\partial}{\partial \theta_i} J(\theta)$ with the **learning rate** γ and the **cost function** $J(\theta)$
- Usually, when training a model, we use a series of N **pairs** $P^{(1)}, \dots, P^{(N)}$ written as $\left[P^{(1)}, \dots, P^{(N)}\right] = \left[\left(x^{(1)}, y^{(1)}\right), \dots, \left(x^{(N)}, y^{(N)}\right)\right]$
- The **cost function** $J(\theta)$ is the average of the losses over the pairs

$$J(\theta) = \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}(\theta)^{(i)} \overset{\text{MSE Loss}}{=} \frac{1}{N} \sum_{i=1}^{N} \left(\hat{y}(\theta)^{(i)} - y^{(i)}\right)^2$$

Recall, that $\hat{y} = f(x, w, b) = f(x, \theta)$. Features x are omitted here for simplicity

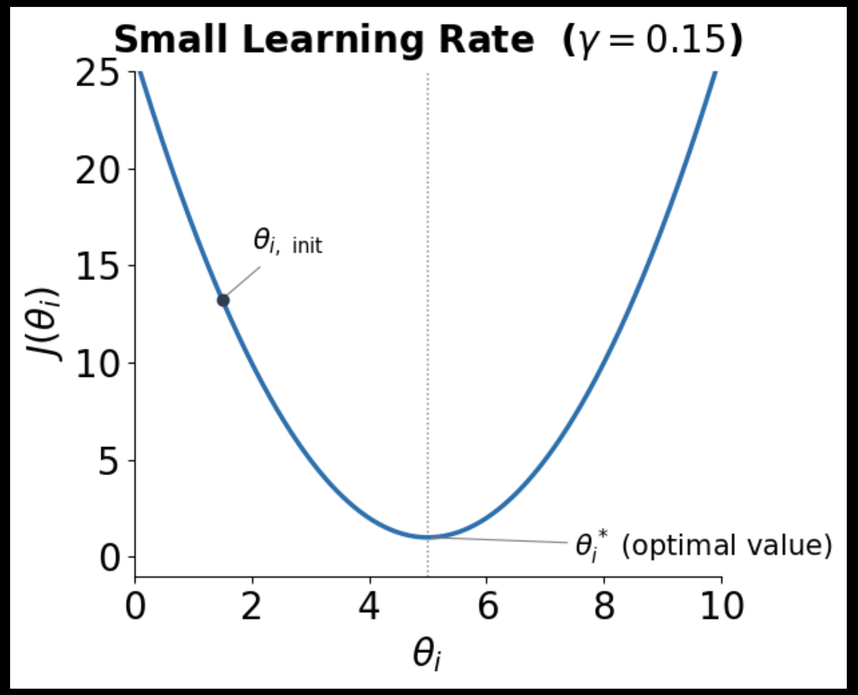
A NOTE ON `optim.SGD`

- Technically, **Stochastic Gradient Descent** refers to updating parameters after each single sample (corresponding to `BATCH_SIZE=1`), but in practice `optim.SGD` is used for all variants of gradient descent:
  - **Full-Batch Gradient Descent** a.k.a. **Batch Gradient Descent** (`BATCH_SIZE=N`)
  - **Mini-Batch Gradient Descent** (`1 < BATCH_SIZE < N`)
  - **Stochastic Gradient Descent** (`BATCH_SIZE=1`)
- The only thing that differs in practice in PyTorch is the batch size specified and passed by the `DataLoader()`

STOCHASTIC GRADIENT DESCENT: EXAMPLE

Update rule: $\theta_{i,\text{new}} = \theta_{i,\text{old}} - \gamma \frac{\partial}{\partial \theta_i} J(\theta)$

Cost function: $J(\theta) \overset{\text{MSE Loss}}{=} \frac{1}{N} \sum_{i=1}^{N} \left(\hat{y}^{(i)} - y^{(i)} \right)^2$

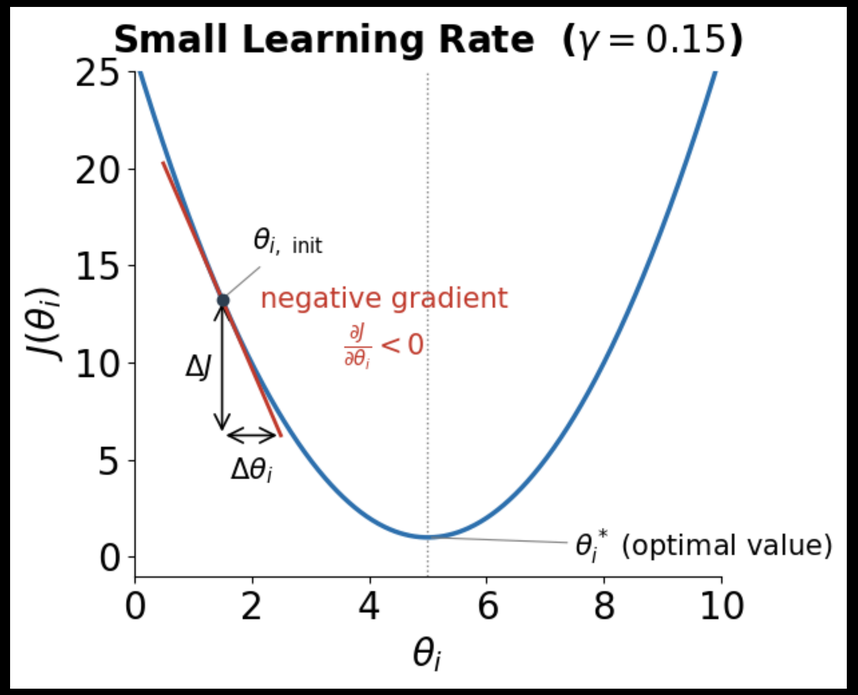


Recall, that $\hat{y} = f(x, w, b) = f(x, \theta)$. Features x are omitted here for simplicity

STOCHASTIC GRADIENT DESCENT: EXAMPLE

Update rule: $\theta_{i,\text{new}} = \theta_{i,\text{old}} - \gamma \frac{\partial}{\partial \theta_i} J(\theta)$

Cost function: $J(\theta) \overset{\text{MSE Loss}}{=} \frac{1}{N} \sum_{i=1}^{N} \left(\hat{y}^{(i)} - y^{(i)} \right)^2$

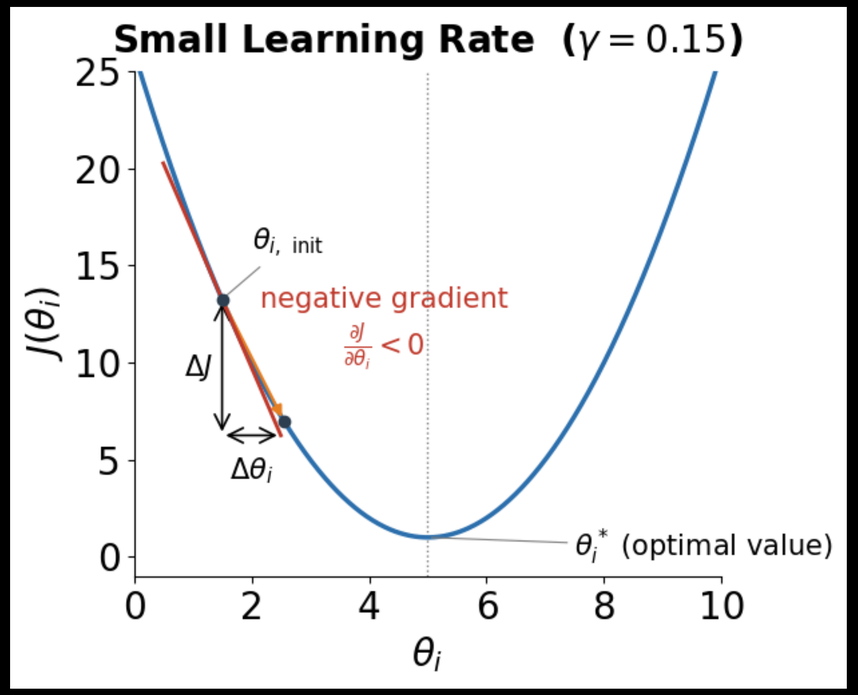


Recall, that $\hat{y} = f(x, w, b) = f(x, \theta)$. Features x are omitted here for simplicity

STOCHASTIC GRADIENT DESCENT: EXAMPLE

Update rule: $\theta_{i,\text{new}} = \theta_{i,\text{old}} - \gamma \frac{\partial}{\partial \theta_i} J(\theta)$

Cost function: $J(\theta) \overset{\text{MSE Loss}}{=} \frac{1}{N} \sum_{i=1}^{N} \left(\hat{y}^{(i)} - y^{(i)} \right)^2$

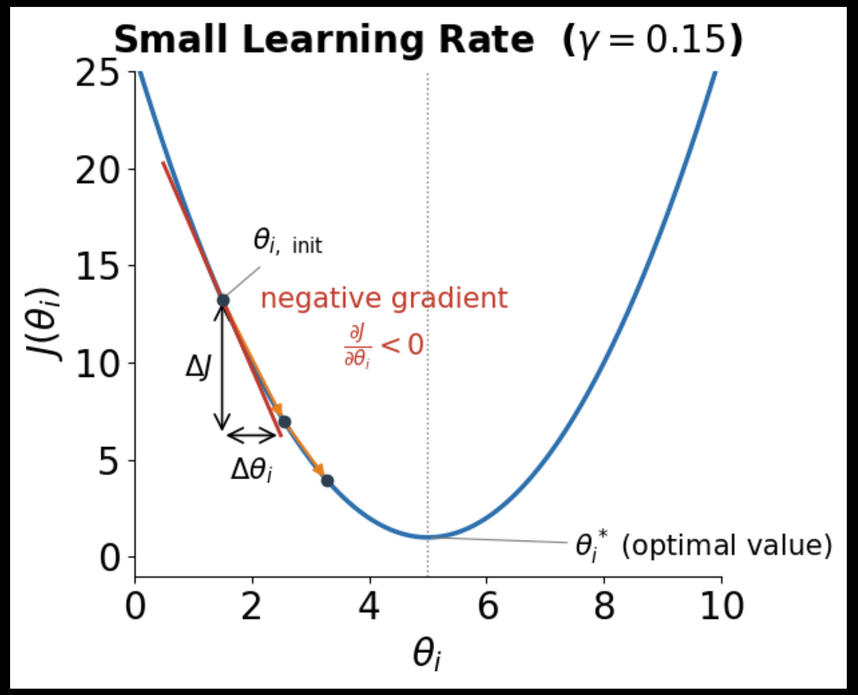


Recall, that $\hat{y} = f(x, w, b) = f(x, \theta)$. Features x are omitted here for simplicity

STOCHASTIC GRADIENT DESCENT: EXAMPLE

Update rule: $\theta_{i,\text{new}} = \theta_{i,\text{old}} - \gamma \frac{\partial}{\partial \theta_i} J(\theta)$

Cost function: $J(\theta) \overset{\text{MSE Loss}}{=} \frac{1}{N} \sum_{i=1}^{N} \left(\hat{y}^{(i)} - y^{(i)} \right)^2$

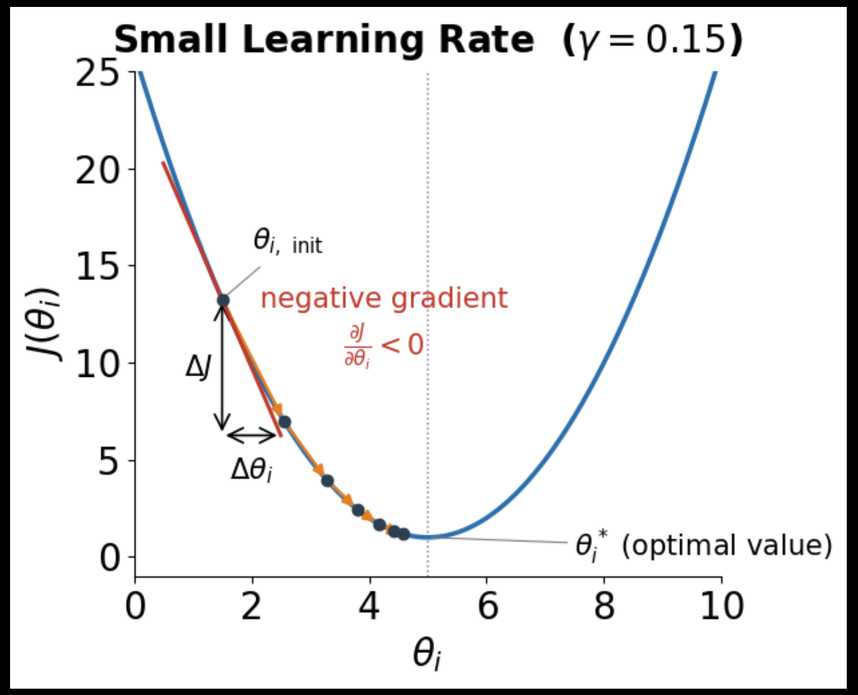


Recall, that $\hat{y} = f(x, w, b) = f(x, \theta)$. Features x are omitted here for simplicity

STOCHASTIC GRADIENT DESCENT: EXAMPLE

Update rule: $\theta_{i,\text{new}} = \theta_{i,\text{old}} - \gamma \frac{\partial}{\partial \theta_i} J(\theta)$

Cost function: $J(\theta) \overset{\text{MSE Loss}}{=} \frac{1}{N} \sum_{i=1}^{N} \left(\hat{y}^{(i)} - y^{(i)} \right)^2$

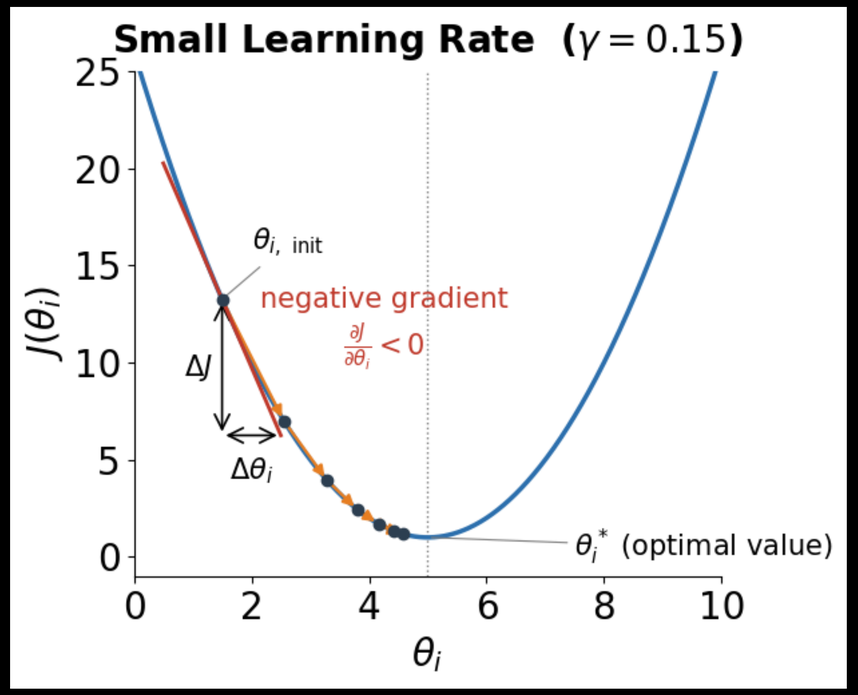


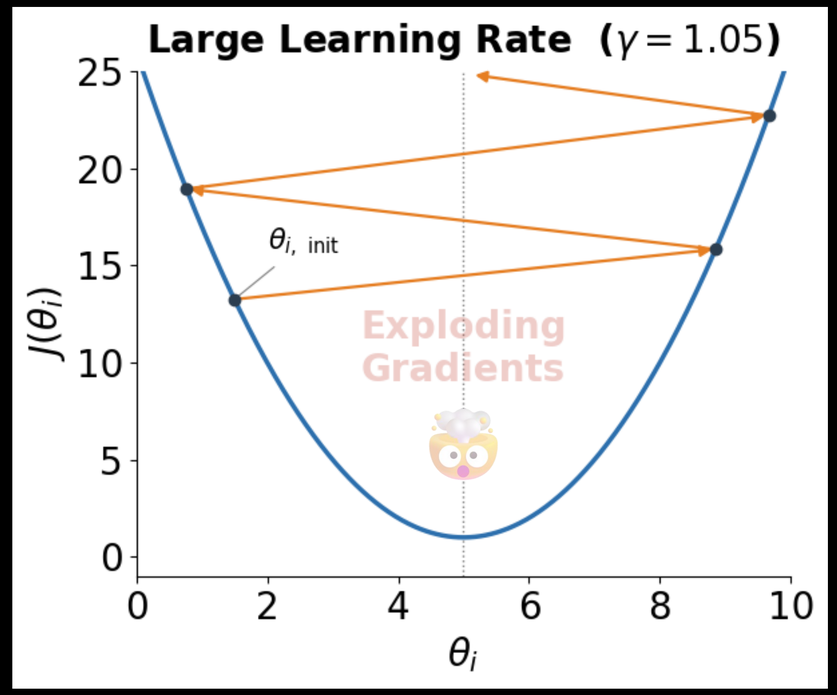
Recall, that $\hat{y} = f(x, w, b) = f(x, \theta)$. Features x are omitted here for simplicity

STOCHASTIC GRADIENT DESCENT: EXAMPLE

Update rule: $\theta_{i,\text{new}} = \theta_{i,\text{old}} - \gamma \frac{\partial}{\partial \theta_i} J(\theta)$

Cost function: $J(\theta) \overset{\text{MSE Loss}}{=} \frac{1}{N} \sum_{i=1}^{N} \left(\hat{y}^{(i)} - y^{(i)} \right)^2$

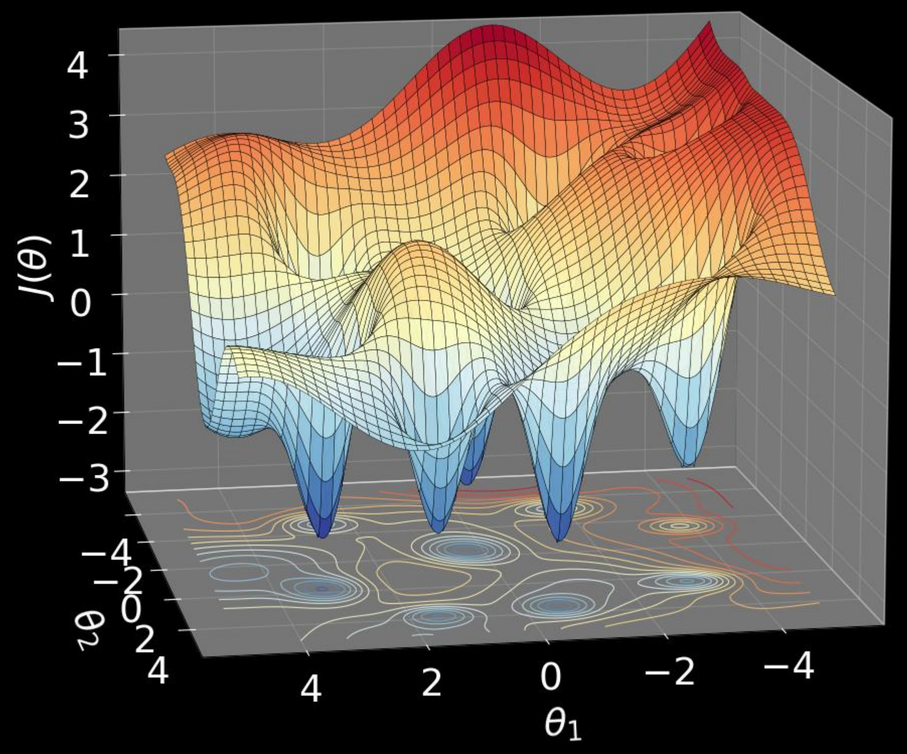




Recall, that $\hat{y} = f(x, w, b) = f(x, \theta)$. Features x are omitted here for simplicity

GRADIENT DESCENT

- Usually, loss surfaces are much more complex, non-convex functions in an extremely high-dimensional space
- The loss surface has one dimension per model parameter θ
  - **<u>We compute one gradient per parameter</u>**
- The plot on the right shows a more realistic 2-dimensional loss surface
- For comparison, the CNN from the assignment had 206,922 parameters



GRADIENT DESCENT: SUMMARY

- Gradients depend on the cost function $J(\theta)$, which is driven by (x, y)
- If we have **N=3200** samples and use **BATCH_SIZE=32** during training, we will use 1% of the data to compute the cost function and gradients and will update the parameters with this limited knowledge
  - **The loss surface for each update depends on the data (y and x) in the batch**
  - The loss curve therefore fluctuates stronger when using batches (**Mini-Batch GD**) ("Drunkard's Walk": *stumbling left and right*, but on average still converging)
  - This also applies to the training, validation and test splits and we should ensure that we have similar distributions resulting in similar loss surfaces for the splits
  - The *stumbling* (noise) of **Mini-Batch GD** prevents overfitting to sharp local minima in the training data, favoring **broader minima** that perform better on validation/test sets (**Full-Batch GD** tends to be too perfect on the training data compromising generalization)
- PyTorch again abstracts the mathematical details of gradient descent away using a feature called **AutoGrad** ("automatic differentiation engine")

SUMMARY: WHAT YOU HAVE LEARNED

- Designing a model consists of **<u>three building blocks</u>**
  - Model, Loss Function and Optimizer
- Training a model requires **<u>five steps</u>**
  - Zero Gradients: `optimizer.zero_grad()`
  - Forward Pass: `output = model(…)`
  - Compute Loss: `loss_fn(output, label)`
  - Backward Pass: `loss.backward()`
  - Parameter Update: `optimizer.step()`
- Use of `Dataset()` and `DataLoader()` for splitting and batching
- Manual and automated feature engineering
- Modular model design & DRY ("Don't Repeat Yourself") principle
- Importance of logging and monitoring to catch bugs and errors

OUTLOOK

- You have covered the fundamentals of PyTorch, experienced some of the common pitfalls and errors and also already discussed some more advanced topics
- The next session will build on that knowledge and should give you a useful toolkit to tackle:
  - advanced data handling (e.g., device management)
  - stable, efficient and reliable training (regularization, hyperparameter tuning, ...)
  - different model architectures (RNNs, GNNs, Transformers)

- **Assignments:** Who will present next?